Project Assignment for SE6003 Cryptography

AY 2026

Nanyang Technological University Singapore
School of Physical and Mathematical Sciences, Division of Mathematical Sciences
Eik List, eiklist@edu.ntu.sg and Frederic Ezerman, fredezerman@edu.ntu.sg

The goal is to read into, understand, and apply or organize a cryptographic topic.

1 Organizational Aspects

1.1 Groups/Registration

The project can be conducted in groups of 1–3 students. Working in groups, instead of individually, is recommended.

1.2 Possible High-level Topics

The project topic can be one of the following:

1. A **survey paper** of a general cryptographic topic, e.g., history of a modern cipher, blockchain, cryptocurrency, multiparty computation, privacy-preserving technologies, public-key Infrastructure, etc., or of a technical cryptographic topic, algorithm, or protocol, e.g, a secret sharing scheme, a key exchange protocol, a digital signature scheme. The algorithm or protocol can be purely classical, or post-quantum, or hybrid.
2. A **software implementation of a cryptographic algorithm** and a documentation of the involved algorithm details and of your implementation.
3. A **software implementation of a cryptographic attack** and a documentation of the involved algorithm details and your implementation.

You can find topic suggestions in Section 2. Each topic can be done by multiple groups, but sharing results across groups is not allowed.

1.3 Outcome and Format

For all topics, including implementation topics, the outcome must include a self-written report in grammatically and orthographically correct English and semantically correct. The report should be a PDF in Springer LNCS format, between 8–12 pages, excluding bibliography.[1]

Report structure. The report must start with the title, a list of authors, and a short abstract that summarizes the main content, taking at most one page. The high-level structure should be:

1. An introduction that motivates the topic. The aim is to be intelligible and understandable by a general audience, e.g., your classmates in this course.
2. A well-organized core part. If a survey is chosen, find a good way to organize the material of the core part, e.g., chronologically, from simple to hard, from high-level to low-level

[1] Templates for LaTeX or Word can be found at https://www.springer.com/gp/computer-science/lncs/conference-proceedings-guidelines.

descriptions, or in another comprehensible form.
If an implementation is chosen, the core part of the report should explain the necessary theoretical background, algorithms, a high-level structure of your implementation, and implementation results, e.g., benchmarks or results from experiments, if any.

3. A closing: The report should close with a discussion, a summary, possible open problems, or a conclusion.
4. The bibliography.
5. Appendices with more results or minor details that are not necessary for understanding the topic. Please keep within the page limit, and structure your report accordingly.

1.4 Implementation Requirements

Implementations must be in Python3 and/or C/C++. The submission must include

- Your source code.
- A build or compile script (a Makefile, a CMakeLists.txt file, or similar build tools).
- A README listing any necessary dependencies, open-source libraries and providing clear instructions for building and running.

C code must compile without errors with gcc/g++ or clang/clang++. Python code must run without errors in Python 3.

You are allowed to use basic modules of the programming language (such as containers, big-integer libraries, numpy, scipy, Boost/STL library, etc.) and open-source libraries. However, you are not allowed to simply borrow existing implementations of cryptographic algorithms without adding significant values, which you must explain clearly. Unless stated otherwise, your implementation need not be particularly efficient or secure.

1.5 Submission

The submission deadline is 23:59 (SGT) Friday, 13 November 2026. Submissions must be **sent via email to fredezerman@ntu.edu.sg**, with the email subject:

`[SE6003 AY2026 Project submission] <last names of the group members>.`

*Start the body of your message with the **full names** and **matriculation numbers** of all group members.* Your submission should include a PDF of your report in the format above. Any further files should be in a zip archive (no other formats, please) and contain **strictly no executable files**.

If submissions would exceed 10 MB, they should come instead as a link to a recognized public temporary service, e.g., a link to a GitHub or gitlab repository, a Dropbox file or folder, an NTU file-sharing service, iCloud, Google Cloud, or similar.

Please double-check before submitting. If you make an error, a single resubmission is allowed, and the last submission before the deadline will be used.

2 Topic Suggestions

The following are exemplary topic suggestions for the purpose of providing you with possible ideas, and for giving you a guideline on the expected level of work and difficulty. Some suggestions may be more or less difficult for different individuals or backgrounds. You are encouraged to make your own suggestions for a topic. For every suggested topic, the given references are only examples. You are expected to research any topic you choose on your own.

2.1 Possible Survey Topics

The following are survey topics. Implementations are not expected, but can be a plus.

(S1) Memory-hard password hashing. Explain memory-hard password hash functions, one of the existing schemes, e.g., scrypt, catena, Argon2, or BalloonHash, the existing security notion of cumulative memory complexity, and how the pebbling game is used to derive security, e.g., for catena. E.g. [1, 10, 17, 26].

(S2) Impact of quantum computers on cryptography. Explain how the availability of quantum computers could allow for breaking existing public-key cryptographic schemes such as RSA or the Diffie-Hellman key exchange. Explain the underlying security assumptions and parameters for classical computers, and RSA, as well as the Diffie-Hellman key exchange in particular. Explain the related quantum algorithms (e.g., Shor's algorithm, Grover's algorithm, or Simon's period-finding algorithm) and why they can solve the underlying problems efficiently. E.g. [33].

(S3) Quantum-resistant cryptography: ML-KEM FIPS 203, ML-DSA FIPS 204, SLH-DSA FIPS 205. Explain the standard scheme of your choice and the underlying mathematics and implementation details in [6, 7, 8], respectively. You can also consider choosing the algorithms that have been marked for standardization but not yet fully standardized, such as **HQC-KEM** [9].

(S4) The Nostradamus attack. Explain the Nostradamus attack on iterated hash functions from [21]. For that, explain the Merkle–Damgård hash-function principle, and Joux's approach for finding multicollisions efficiently [20].

(S5) Differences between Bitcoin and alternative ledgers. Explain the broad concept of blockchain, the working and the components of Bitcoin, and the differences between Bitcoin, Ethereum [12], and Ripple [2].

(S6) Other topics of your choice.

2.2 Possible Topics for Implementing Cryptographic Algorithms

The following topics are implementation topics for cryptographic modes, schemes, or protocols. All implementation-related aspects from above apply here.

(I1) Memory-hard password-hashing functions. Explain and implement a proof of concept of one of the following memory-hard password hashing schemes: scrypt [25], catena [17], or BalloonHash [10]. You do not have to implement the internal hash function (e.g., SHA-2) yourselves.

(I2) RSA-OAEP. Explain and implement a proof of concept of RSA-OAEP with SHA-2 or SHA-3, including prime-factor sampling, and the Miller-Rabin prime-number test. You do not have to implement SHA yourselves, but you do have to implement RSA and all

aspects involved, such as padding. E.g. [4, 27, 28].

(I3) Diffie-Hellman key exchange. Explain and implement the Diffie-Hellman key exchange with flexible-size prime numbers for at least up to 2048-bit prime numbers, including the square-and-multiply algorithm for exponentiation, the Miller-Rabin prime- number test, and hashing. You do not have to implement a used hash functions yourselves.

(I4) Authenticated Encryption with AES. Explain and implement encryption and decryption in C with AES-NI and CLMUL instructions of AES-GCM-128. E.g. [16, 18, 19].

(I5) X-Wing. Explain and implement an example of a general-purpose hybrid post-quantum KEM [22].

(I6) Other topics of your choice.

2.3 Possible Cryptanalysis Implementation Topics

The following topics are implementation topics of cryptanalytic attacks. All implementation-related aspects from above also apply here. It is expected that your implementation has a significant success probability for recovering certain parameters (a secret key, a hash value, a prime factor, etc.) that are chosen anew and randomly for each run. You can involve test cases with hard-coded test parameters.

(C1) Differential cryptanalysis of round-reduced AES. Explain and implement the attack on 5-round AES-128 from [3]. You do not have to implement AES yourselves.

(C2) Integral cryptanalysis of round-reduced AES. Explain and implement the integral attack on 5-round AES from [30]. You do not have to implement AES yourselves.

(C3) Meet-in-the-middle attack on Textbook RSA. Explain and implement the meet-in-the-middle attack on RSA with at least 30-bit prime numbers. E.g. [11, Section 1.2].

(C4) A basic number-field sieve. Explain and implement a basic number-field sieve that can recover the factors of at least 60-bit RSA values $n = p \cdot q$ that are products of 30-bit prime factors p and q. E.g. [15, 23].

(C5) Pollard's Rho for collisions. Explain and implement a collision search with Pollard's rho that can find collisions for at least 50-bit truncated hashes (e.g., they collide on the first or last 50 bits) for SHA-2 or SHA-3. Then, explain factorization of large numbers with Pollard's rho, and implement a factorization for at least 60-bit random numbers. You do not have to implement SHA yourselves. E.g. [29].

(C6) Distinguished-point collision search. Explain and implement a small distinguished-point collision search and implement the search to find collisions for 50-bit (truncated) hashes for SHA-2 or SHA-3. You do not have to implement SHA yourselves. E.g. [31, 32].

(C7) Recovering neural-network models. Explain and implement a proof of concept of the approach in one of the following works to recover the weights (between 0.0 and 1.0 each) of a small three-layer network with $(10, 10, 1)$ nodes: [13, 14, 24].

(C8) Other topics of your choice.

2.4 Referencing

Your report should include a bibliography, referencing the sources your report referred to. Additional sources can be provided as footnotes. Note that:

- All text, figures, source code, diagrams, quotations, or similar material used in your report or implementation that was not authored by you must be properly cited to the original authors, which must be attributable to accountable human or legal persons (firms, institutions).
- Machine-generated media is not an accountable reference for facts. Anonymous compilations without clear accountable authors (e.g., reddit, wikipedia, etc.) are not accountable references for facts (you can use them for opinions). If you use non-accountable references, it is your task to trace down the correct original source to accountable references.
- For books and academic works, e.g., `scholar.google.com` and `dblp.org` can provide citation information. When citing websites, please indicate the date of your most recent access.
- For your research, you are allowed to use any sources that satisfy the ethical standards of NTU. For the report, you are allowed to use any tools for spell checking, generating figures, diagrams, plots, tables, etc., that satisfy the ethical standards of NTU. For your implementations, you are allowed to use any tools for testing, linting, organizing, and suggesting code that satisfy the ethical standards of NTU.
- All tool-generated text must be clearly declared as such, and the used tools and the specific instances of use must be clearly named.
- All members of the group, to equal shares, are responsible for everything in their report and every piece of source code in their implementations that is not indicated as being authored by accountable third-party authors. Each part of the report and each part of source code not declared as authored by third parties must be fully understood and explainable by at least one member of the group if asked.

References

[1] Joël Alwen and Jeremiah Blocki. Efficiently computing data-independent memory-hard functions. In Matthew Robshaw and Jonathan Katz, editors, *CRYPTO II*, volume 9815 of LNCS, pages 241–271. Springer, 2016. `https://doi.org/10.1007/978-3-662-53008-5_9`.

[2] Frederik Armknecht, Ghassan O. Karame, Avikarsha Mandal, Franck Youssef, and Erik Zenner. Ripple: Overview and Outlook. In Mauro Conti, Matthias Schunter, and Ioannis G. Askoxylakis, editors, *TRUST 2015*, volume 9229 of LNCS, pages 163–180. Springer, 2015. `https://doi.org/10.1007/978-3-319-22846-4_10`.

[3] Achiya Bar-On, Orr Dunkelman, Nathan Keller, Eyal Ronen, and Adi Shamir. Improved key recovery attacks on reduced-round AES with practical data and memory complexities. *J. Cryptol.*, 33(3):1003–1043, 2020. `https://doi.org/10.1007/s00145-019-09336-w`.

[4] Mihir Bellare and Phillip Rogaway. Optimal asymmetric encryption. In Alfredo De Santis, editor, *EUROCRYPT*, volume 950 of LNCS, pages 92–111. Springer, 1994. `https://doi.org/10.1007/BFb0053428`.

[5] Daniel J. Bernstein, Andreas Hülsing, Stefan Kölbl, Ruben Niederhagen, Joost Rijnveld, and Peter Schwabe. The SPHINCS+ Signature Framework. In *ACM CCS*, pages 2129–2146. ACM, 2019. `https://doi.org/10.1145/3319535.3363229`.

[6] Federal Information Processing Standards Publication, FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard, National Institute of Standards and Technology, August 2024. `https://csrc.nist.gov/pubs/fips/203/final`.

[7] Federal Information Processing Standards Publication, FIPS 204: Module-Lattice-Based Digital Signature Standard, National Institute of Standards and Technology, August 2024. https://csrc.nist.gov/pubs/fips/204/final.

[8] Federal Information Processing Standards Publication, FIPS 205: Stateless Hash-Based Digital Signature Standard, National Institute of Standards and Technology, August 2024. https://csrc.nist.gov/pubs/fips/205/final.

[9] P. Gaborit. C. Aguilar-Melchor, N. Aragon, S. Bettaieb, L. Bidoux, O. Blazy, J-. Deneuville, E. Persichetti, G. Zémor, J. Bos, A. Dion, J. Lacan, J-M. Robert, P. Véron, P. L. Barreto, S. Ghosh, S. Gueron, T. Güneysu, R. Misoczki, J. Richter-Brokmann, N. Sendrier, J-P. Tillich, and V. Vasseur, Hamming Quasi-Cyclic (HQC), Aug. 2025. https://pqc-hqc.org/doc/hqc_specifications_2025_08_22.pdf

[10] Dan Boneh, Henry Corrigan-Gibbs, and Stuart E. Schechter. Balloon hashing: A memory-hard function providing provable protection against sequential attacks. In *ASIACRYPT I*, volume 10031 of LNCS, pages 220–248, 2016. https://doi.org/10.1007/978-3-662-53887-6_8.

[11] Dan Boneh, Antoine Joux, and Phong Q. Nguyen. Why textbook ElGamal and RSA encryption are insecure. In *ASIACRYPT*, volume 1976 of LNCS, pages 30–43. Springer, 2000. https://doi.org/10.1007/3-540-44448-3_3.

[12] Vitalik Buterin. A next-generation smart contract and decentralized application platform. Technical report, Ethereum Foundation, 2014. https://ethereum.org/content/whitepaper/whitepaper-pdf/Ethereum_Whitepaper_-_-Buterin_2014.pdf.

[13] Nicholas Carlini, Jorge Chávez-Saab, Anna Hambitzer, Francisco Rodríguez-Henríquez, and Adi Shamir. Polynomial time cryptanalytic extraction of deep neural networks in the hard-label setting. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT I*, volume 15601 of Lecture Notes in Computer Science, pages 364–396. Springer, 2025. https://doi.org/10.1007/978-3-031-91107-1_13.

[14] Nicholas Carlini, Matthew Jagielski, and Ilya Mironov. Cryptanalytic extraction of neural network models. In *CRYPTO III*, volume 12172 of LNCS, pages 189–218. Springer, 2020. https://doi.org/10.1007/978-3-030-56877-1_7.

[15] Stefania Cavallar, Bruce Dodson, Arjen K. Lenstra, Paul C. Leyland, Walter M. Lioen, Peter L. Montgomery, Brian Murphy, Herman J. J. te Riele, and Paul Zimmermann. Factorization of RSA-140 using the number field sieve. In Kwok-Yan Lam, Eiji Okamoto, and Chaoping Xing, editors, *ASIACRYPT*, volume 1716 of Lecture Notes in Computer Science, pages 195–207. Springer, 1999. https://doi.org/10.1007/978-3-540-48000-6_16.

[16] Morris J. Dworkin. SP 800-38D: Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC. NIST, 2007. https://doi.org/10.6028/NIST.SP.800-38D.

[17] Christian Forler, Stefan Lucks, and Jakob Wenzel. Memory-demanding password scrambling. In *ASIACRYPT II*, volume 8874 of LNCS, pages 289–305. Springer, 2014. https://doi.org/10.1007/978-3-662-45608-8_16.

[18] Shay Gueron. Intel Advanced Encryption Standard (AES) New Instructions Set. Intel Corporation, 2010. https://www.intel.com/content/dam/doc/white-paper/advanced-encryption-standard-new-instructions-set-paper.pdf.

[19] Shay Gueron and Michael E. Kounavis. Intel Carry-less Multiplication Instruction and Its Usage for Computing the GCM Mode. Intel Corporation, 2010.

[20] Antoine Joux. Multicollisions in iterated hash functions. Application to cascaded constructions. In *CRYPTO 2004*, volume 3152 of LNCS, pages 306–316. Springer, 2004. https://doi.org/10.1007/978-3-540-28628-8_19.

[21] John Kelsey and Tadayoshi Kohno. Herding Hash Functions and the Nostradamus Attack. In *EUROCRYPT 2006*, volume 4004 of LNCS, pages 183–200. Springer, 2006. https://doi.org/10.1007/11761679_12.

[22] Manuel Barbosa, Deirdre Connolly, João Diogo Duarte, Aaron Kaiser, Peter Schwabe, Karolin Varner, and Bas Westerbaan, X-Wing: The Hybrid KEM You've Been Looking For. IACR Communications in Cryptology, vol. 1, no. 1, Apr 09, 2024. https://cic.iacr.org/p/1/1/21

[23] Arjen K. Lenstra, Hendrik W. Lenstra Jr., Mark S. Manasse, and John M. Pollard. The number field sieve. In *ACM STOC*, pages 564–572. ACM, 1990. https://doi.org/10.1145/100216.100295.

[24] Isaac Andrés Canales Martinez, Jorge Chávez-Saab, Anna Hambitzer, Francisco Rodríguez-Henríquez, Nitin Satpute, and Adi Shamir. Polynomial time cryptanalytic extraction of neural network models. In Marc Joye and Gregor Leander, editors, *EUROCRYPT III*, volume 14653 of Lecture Notes in Computer Science, pages 3–33. Springer, 2024. https://doi.org/10.1007/978-3-031-58734-4_1.

[25] Colin Percival. Stronger key derivation via sequential memory-hard functions, 2009. https://www.tarsnap.com/scrypt/scrypt.pdf.

[26] Colin Percival and Simon Josefsson. The scrypt Password-Based Key Derivation Function, August 2016. https://www.rfc-editor.org/info/rfc7914.

[27] David Pointcheval. How to encrypt properly with RSA. *CryptoBytes*, 5(1):10–19, 2002.

[28] David Pointcheval. OAEP: optimal asymmetric encryption padding. In *Encyclopedia of Cryptography and Security*. Springer, 2005. https://doi.org/10.1007/0-387-23483-7_284.

[29] J. M. Pollard. A Monte Carlo method for factorization. *BIT*, 15:331–334, 1975. https://doi.org/10.1007/BF01933667.

[30] Yosuke Todo and Kazumaro Aoki. FFT key recovery for integral attack. In *CANS*, volume 8813 of LNCS, pages 64–81. Springer, 2014. https://doi.org/10.1007/978-3-319-12280-9_5.

[31] Paul C. van Oorschot and Michael J. Wiener. Parallel collision search with application to hash functions and discrete logarithms. In *ACM CCS*, pages 210–218. ACM, 1994. https://doi.org/10.1145/191177.191231.

[32] Paul C. van Oorschot and Michael J. Wiener. Parallel collision search with cryptanalytic applications. *J. Cryptol.*, 12(1):1–28, 1999. https://doi.org/10.1007/PL00003816.

[33] Hiu Yung Wong. Shor's Algorithm, pages 289–298. Springer International Publishing, Cham, 2024. https://doi.org/10.1007/978-3-031-36985-8_29.